

Tema 10: Algoritmos Probabilistas

Introducción al tema

El capítulo 10 introduce una categoría particular de algoritmos cuyo rasgo distintivo es el uso explícito de números aleatorios para influir en su ejecución. A estos algoritmos se los denomina **probabilistas**, porque su comportamiento (ya sea el tiempo de ejecución o la corrección del resultado) depende, parcial o totalmente, de decisiones aleatorias realizadas durante su ejecución.

Los algoritmos probabilistas se justifican porque permiten resolver problemas complejos de una manera más eficiente, o incluso resolver problemas que mediante métodos deterministas serían imposibles o altamente costosos en términos de tiempo computacional.

Definición de algoritmo probabilista

Un algoritmo probabilista es aquel que, además de recibir una entrada ordinaria, tiene acceso a una fuente de números aleatorios. Por tanto, dos ejecuciones de un mismo algoritmo sobre la misma entrada pueden tener resultados o tiempos de ejecución diferentes, dependiendo de los valores aleatorios generados durante la ejecución.

Tipología de algoritmos probabilistas

El texto explica dos grandes clases de algoritmos probabilistas que se diferencian en su criterio de éxito y eficiencia:

1. Algoritmos de tipo Monte Carlo

- Estos algoritmos siempre finalizan en un tiempo determinado (es decir, son de tiempo acotado), pero su resultado puede ser incorrecto con una probabilidad baja y predefinida.
- Sin embargo, es posible disminuir esta probabilidad de error aumentando la cantidad de pruebas aleatorias durante la ejecución.
- Son muy usados en problemas donde la verificación del resultado es rápida, pero encontrar la solución exacta mediante métodos deterministas es demasiado costoso.

- **Ejemplo clave:** *Prueba de primalidad* como el **test de Miller-Rabin**, donde se puede determinar si un número es primo con un margen de error arbitrariamente bajo.

2. Algoritmos de tipo Las Vegas

- En esta categoría, el algoritmo siempre produce un resultado correcto, pero el tiempo de ejecución no es fijo, ya que depende de las elecciones aleatorias realizadas durante la ejecución.
- No existe margen de error, pero no se garantiza una duración exacta para finalizar.
- **Ejemplo clásico:** *Ordenación mediante QuickSort aleatorizado*, donde el pivote se elige al azar y el algoritmo siempre ordena correctamente, aunque el número de comparaciones puede variar de una ejecución a otra.

Ejemplos explicados en el tema

a) Pruebas probabilísticas de primalidad

El texto desarrolla la lógica detrás de pruebas como la de Miller-Rabin. Estas pruebas no determinan con certeza si un número es primo, pero pueden asegurar que si el número no pasa la prueba, definitivamente es compuesto, y si la pasa varias veces, es casi seguramente primo.

b) Ordenación aleatoria

Explica el uso de QuickSort con pivotes elegidos aleatoriamente, lo que estadísticamente mejora el tiempo promedio de ejecución y reduce la probabilidad de encontrar los peores casos típicos del algoritmo clásico.

c) Selección aleatoria (Randomized-Select)

También se trata el problema de encontrar el k-ésimo menor elemento de una lista desordenada. El uso de aleatorización permite resolver este problema con un tiempo de ejecución promedio lineal $O(n)$, mejorando respecto a métodos deterministas clásicos.

Aplicaciones prácticas

El texto menciona varias áreas donde los algoritmos probabilistas se aplican con éxito:

- Criptografía (pruebas de primalidad, generación de claves)

- Búsqueda eficiente en grandes bases de datos
- Algoritmos de grafos
- Problemas numéricos donde la solución determinista es computacionalmente inviable

Además, subraya que en contextos donde se requiere una respuesta rápida y "suficientemente buena", los algoritmos probabilistas son preferibles a sus contrapartes deterministas.

Ventajas resaltadas

- Mayor eficiencia promedio frente a algoritmos clásicos.
- Implementaciones más sencillas para problemas complejos.
- Alta aplicabilidad a problemas reales donde no se requiere perfección absoluta.
- Capacidad de ajustar el balance entre tiempo y precisión según las necesidades.

Limitaciones señaladas

- Posibilidad de errores (en algoritmos de tipo Monte Carlo).
- Variabilidad del tiempo de ejecución (en algoritmos Las Vegas).
- Análisis matemático más complejo debido al componente probabilístico.
- Requiere fuentes confiables de números aleatorios.